Computational Complexity

Reading guide and Homework for **Week 02**

1 Reading Guide

In the second class we have learned:

- How to measure the amount of time taken by a Turing machine, and what is how this gives us a notion of *time complexity* of a function $f : \{0,1\}^* \to \{0,1\}^*$. You should review this in §0.3 (Big-O Notation), §1.3 and §1.6 of the book.
- The time complexity classes: P, E, EXP, and the *time hierarchy theorem* showing that these classes are different. Read §3.1.
- We saw several examples of problems that are in P, and of problems that don't obviously appear to be in P, including some that actually are.
- We learned about exhaustive search algorithms, and why they are not efficient. We learned about Gödel's problem and Nash's problem. This is not in the book, but at the end of your reading, you should read the related section §2.7, and the "Chapter Notes and History" section that follows it.
- We talked about efficiently-universal multitape Turing machines, time-constructible functions, and proved the time hierarchy theorem.

There were several details that we covered very quickly, and which **must** be looked at carefully, in order to properly understand what is going on. This kind of familiarity will be necessary to understand what's being said in class.

2 Homework Exercises

From now on, exercises are organized by the skill that is being trained. The test and exam questions will always be about testing one of the skills appearing in the homework.

Skill: *Understanding the big-O notation.*

1. Prove or disprove the following:
   (a) For all $d > c \geq 0$, $n^c = o(n^d)$.
   (b) For all $c \geq 0$ and $d > 1$, $n^c = o(d^n)$.
   (c) If $f = O(n)$ and $g = \Theta(n)$, then
      i. $f(n) + g(n) = O(n)$.
      ii. $f(n)g(n) = O(n^2)$.
      iii. $f(n) - g(n) = O(1)$.
      iv. $\frac{f(n)}{g(n)} = O(1)$.

Skill: *show that a given problem has an efficient algorithm (i.e. that it is in* P*).*

2. An n-vertex graph is called d-regular if every vertex has degree exactly d (i.e. has d incident edges).

   Show that, for any fixed natural number $d \geq 1$, the problem of deciding whether a given n-vertex graph is d regular is in P.

   Then show that the problem of deciding whether G is d-regular, when given an n-vertex graph G and a number $d \in \{1, \ldots, n-1\}$ as input (so d is not fixed), is also in P.

3. A k-clique in an n-vertex undirected graph $G = ([n], E)$ is a set $S \subseteq [n]$ of $|S| = k$ vertices of G such that any pair of vertices in S has an edge between them ($\forall_{a,b \in S, a \neq b} \{a, b\} \in E$).

   Show that, for any fixed natural number $k \geq 2$, the problem of deciding whether a given n-vertex graph G has a k-clique is in P.

   How about the problem of deciding whether G has a k-clique, when given both G and k as input (so k is not fixed)? (just think about it, more below.)

4. Consider the problem where we are given a list $X = (x_1, \ldots, x_n)$ of natural numbers, where each x_i is bounded as $0 \leq x_i \leq n^3$, and we wish to *sort* the list, i.e., output a list Y with the same numbers, (Including multiplicities, i.e., if a number appears repeated k times in X, it should also appear k times in Y) but sorted in ascending order. Show that this problem can be solved in time $O(n \log n)$ on a multitape Turing machine (hint: lookup *merge sort*).

5. The $\mathsf{3SUM}$ problem is as follows: you are given as input a list $(x_1, \ldots, x_n)$ of natural numbers, with each $-n^3 \leq x_i \leq n^3$. The goal is to decide whether there exist three indices $i, j, k \in [n]$ such that $x_i + x_j + x_k = 0$.

   Show that $\mathsf{3SUM}$ is in P.

6. Show that $\mathsf{3SUM}$ can be solved in time $O(n^2 \log n)$ by a RAM Turing machine (we defined it in class, it also appears in the book's Exercise 1.9).

7. Given two strings $a, b \in \Gamma^*$ a common subsequence of a and b is a sequence of symbols which is a subsequence of both a and b. For example, *abara* is a common subsequence of *abracadabra* and *barbarian*. Give an algorithm which outputs the maximal length of a common subsequence of two strings in polynomial time.